

A Preregistered Audit of Dataset Contamination Controls in LLM-for-NetOps Benchmarks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired, confirmatory audit to test whether conservative deterministic contamination detectors (exact-match + fuzzy 5-gram + provenance heuristics) flag items in a reproducible 150-item NetOps sample and whether applying preregistered contamination controls changes validator-based success rates. Crucially, the executed sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") and shared generation semantics (independent_condition_generation = false), recorded in the archived model and task manifests; these two execution facts materially limited the audit’s sensitivity to generation-level contamination. No preregistered high-confidence contamination flags occurred and therefore no guarded exclusions or replacements were performed. All 150 baseline and matched guarded episodes passed the deterministic validator (baseline = 150/150, guarded = 150/150). The paired success-rate difference = 0.0 percentage points (95% CI [0.0, 0.0]); exact McNemar two-sided $p = 1.0$. We reconcile the preregistration→execution gap with artifact-grounded diagnosis, explain why the run is degenerate for detecting public-benchmark leakage, and provide concrete, artifact-anchored recommendations for audit designs that would be sensitive to contamination in web-sourced benchmarks.

I. INTRODUCTION

Motivation. Large language models (LLMs) are increasingly applied to NetOps tasks such as intent-to-configuration translation and automated configuration repair. Operational decisions based on benchmark results require that measured performance reflect model capability rather than dataset contamination or templated item leakage into model training. Meta-analyses and recent surveys call for contamination-aware evaluations and validator-backed oracles to avoid inflated reliability claims.

Research question and contribution. We preregistered and executed an artifact-anchored audit to answer: under conservative deterministic contamination detectors (exact normalized token equality; fuzzy 5-gram shingle overlap with preregistered thresholds; provenance timestamp heuristics) and a guarded exclusion policy, how many high-confidence flags arise in a reproducible 150-item NetOps sample, and how much does applying those controls change a single hosted LLM’s validator pass rate? Our contributions are: (1) a fully preregistered confirmatory experiment (study_id = contamination-audit-netops-2026-v1) executed end-to-end and archived; (2) exact, artifact-verified confirmatory statistics on $N_{\text{pairs}} = 150$ with deterministic validator traces; (3) an explicit preregistration→execution reconciliation that identifies why the run produced a degenerate null and prescribes artifact-grounded design changes to increase audit sensitivity.

Scope and bounded claims. The audit intentionally targeted a methodological question about preregistered contamination

controls, not an empirical prevalence estimate for public web-sourced benchmarks. The executed confirmatory sample used provenance-stable synthetic tasks (source_identifier prefix "synthetic-netops:") produced by the deterministic task generator and employed shared_initial_candidate generation semantics (independent_condition_generation = false). Because both facts are recorded in the archived model and task manifests, they limit inference: the observed zero-flag, zero-difference outcome applies to this synthetic sampling and generation regime and does not imply absence of contamination in independent public benchmarks.

II. RELATED WORK

LLM-driven NetOps systems and verifier integration. Recent system and evaluation work has investigated LLMs for intent-driven configuration synthesis and for proposing configuration repairs; these studies highlight practical value but also recurring failure modes that motivated tool-assisted or agentic architectures. Representative intent→configuration systems show that LLMs can produce syntactically valid device artifacts when guided by structured prompts or in-context examples, but that semantic correctness (reachability, policy preservation) typically requires an external validator or iterative refine-and-repair loop [<https://openalex.org/W4404602270>, 10.1109/vtc2025-fall65116.2025.11310185]. Benchmarks and controlled experiments on LLM-assisted repair report improved average repair quality when the pipeline integrates a deterministic checker (or formal verifier) with generation and repair steps; however, per-case regressions remain common and depend strongly on the verifier’s property coverage and on dataset realism [<https://openalex.org/W7157506260>, <https://openalex.org/W7163636922>].

Formal verification and deterministic oracles. The NetOps literature provides mature deterministic verification techniques (Header Space Analysis and derived tools, localized subspecification approaches, and NetKAT/related formalizations) that are practical oracles for reachability and many policy properties; these tools form the empirical basis for validator-backed evaluation in NetOps generate-validate pipelines [<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>, <https://openalex.org/W2742515467>, <https://openalex.org/W1882012874>]. Studies integrating such verifiers show that the choice of checks (which fields and temporal/workflow constraints are modeled) materially

changes which generation errors are visible to the auditor—an observation we incorporate by using a deterministic validator (deterministic_netops_validator_v5) that emits parse→state traces and explicit violation codes for every episode.

Agentic validate–repair architectures and their limits. Multiple contemporaneous projects evaluate agentic or multi-stage designs where an LLM proposes a candidate, a verifier checks it, and a repair generator attempts fixes when violations are found; these generate–validate–repair patterns generally improve mean repair rates versus single-pass generation but do not eliminate regressions and depend on the repair-policy and localization strategy [https://openalex.org/W7162218766, https://openalex.org/W7163636922]. Empirical studies also show scale sensitivity: effectiveness reported on curated or synthetic misconfiguration tasks often degrades when confronted with more heterogeneous, multi-vendor, or production-sourced corpora. These findings motivate two audit recommendations we make below: (1) use provenance-rich sampling to exercise contamination detectors, and (2) persist richer validator/tracer outputs to analyze subtle regression modes.

Benchmark validity, contamination, and detector strategies. Recent surveys and meta-evaluations emphasize two recurring evaluation validity risks: dataset contamination, where benchmark items (or templated prompts) are present in LLM training data and inflate measured accuracy, and construct invalidity, where tasks are overly templated or low complexity and fail to exercise operational difficulty [https://openalex.org/W4404534210, https://openalex.org/W7166900660]. The literature recommends deterministic fingerprinting (canonicalized exact matching), fuzzy n-gram overlap, and provenance/timestamp tracing as complementary heuristic detectors; these techniques form the basis for many proposed audit workflows because they are computationally tractable and interpretable. However, prior work also highlights tradeoffs: exact fingerprints miss paraphrased memorization; n-gram overlap thresholds require sensitivity analysis; and provenance heuristics depend on available source metadata and archival coverage. Our preregistration implemented a conservative combination (exact normalized equality OR fuzzy 5-gram overlap ≥ 0.80 for high confidence, with lower thresholds for sensitivity analyses) precisely because the literature documents these tradeoffs and recommends multimodal flagging and post-hoc adjudication [https://openalex.org/W4385681611, https://openalex.org/W7166900660].

How our study differs from and complements prior work. Unlike broader capability benchmarks that measure absolute LLM performance across many models, or agentic repair evaluations that emphasize mean repair rates, our study is a preregistered contamination-control audit: it asks whether a conservative, preregistered detector ensemble would produce high-confidence flags in a reproducible 150-item sample and whether deterministic guarded exclusions would alter validator pass rates. Prior benchmark and system papers typically do not publish preregistered exclusion

rules or the exact generation semantics used for paired comparisons; that omission can obscure whether observed generation discordance arises from detector sensitivity or from sampling/generation choices. By freezing detector thresholds, exclusion policies, sampling seeds, and generation semantics in a preregistration and archiving the validator traces, our experiment complements the literature by making the audit contract explicit and by demonstrating (artifact-grounded) how deterministic sampling and shared generation semantics can collapse a paired confirmatory test into a degenerate outcome. This reconciliation—linking verifier traces, provider call accounting, and the detector workflow to the resulting paired contingency—is the primary methodological contribution that complements prior empirical and system-level findings [https://openalex.org/W7157506260, https://openalex.org/W7162218766, https://openalex.org/W4404602270].

III. METHODOLOGY

Overview and preregistration. The experiment was preregistered (study_id = contamination-audit-netops-2026-v1) and froze the confirmatory estimand, exclusion rules, detectors, replacement policy, sampling plan, and analysis executor prior to observing any model outputs. The primary estimand is the paired success-rate difference (guarded - baseline) across item×model pairs. The analysis executor is paired_binary_analysis_v1 and the preregistration specifies an exact McNemar two-sided primary test plus a paired bootstrap percentile 95% confidence interval (10,000 resamples, bootstrap_seed = 1729). Planned confirmatory accounting: task_count = 150 items, planned_episode_count = 300 episodes (baseline + guarded), maximum_model_calls = 300.

Sampling and deterministic generation. To ensure reproducibility the preregistration allowed deterministic synthetic task generation. The executed run used deterministic_netops_task_generator_v5 to produce the confirmatory sample of 150 items (task_count = 150; execution artifact task_manifest.jsonl records per-task generation_seed values). Each generated task includes a source_identifier and task_payload that encode initial_state, expected_final_state, allowed_touched_fields, explicit_required_sequence, and a textual intent. The preregistration required fixed seeds and deterministic selection for replacements; the execution artifacts (execution/task_manifest.jsonl and representative task entries) show per-task generator seeds (e.g., generation_seed: 1..N in representative examples) and the source_identifier prefix "synthetic-netops:" indicating provenance-stable synthetic items in this run.

Generation semantics and effect on independent trials. The preregistration permitted both independent generation per condition and shared generation semantics; it specified independent_condition_generation as a parameter. The executed model_configuration.json sets independent_condition_generation = false (shared_initial_candidate semantics). Under shared_initial_candidate semantics the pipeline performs a single model generation per item and

reuses that shared candidate for both baseline and guarded episodes unless an exclusion/replacement rule is triggered. This execution choice reduces hosted model calls (executed `model_calls_used = 150`) relative to the preregistered maximum (300) and mechanically collapses baseline vs guarded discordance when no exclusions occur. The execution manifest and deterministic_reconciliation artifacts record these counts (`planned_episode_count = 300`, `completed_episode_count = 300`, `model_calls_used = 150`) and confirm that provider calls were counted under a single shared generation stage per item.

Contamination detectors (implementation and thresholds). The preregistered detector suite implemented three deterministic heuristics: 1) Exact normalized token equality: canonicalize whitespace, normalize tokens, and flag exact matches between item text and archived public corpus fingerprints (high confidence on equality). 2) Fuzzy 5-gram shingle overlap: compute 5-gram shingles on canonicalized tokens and measure overlap/Jaccard; preregistered thresholds flagged high confidence when overlap ≥ 0.80 , and lower confidence brackets at 0.65 and 0.50 for sensitivity analyses. 3) Provenance timestamp heuristics: compare item timestamps and DOI/URL metadata to public archive timestamps; flag items that predate the model public cutoff per heuristics in the preregistration.

Guarded policy and replacements. The preregistered guarded policy deterministically excludes only high-confidence flagged items (exact match OR 5-gram overlap ≥ 0.80). Excluded items are replaced by deterministic retemplating and topology augmentation drawn from the same task generator with fixed augmentation seeds; replacements must preserve intent similarity by an embedding safeguard (preregistered minimal cosine similarity threshold: cosine ≥ 0.85) before acceptance. Replacement artifacts, retemplating traces, and embedding similarity checks are included in the preregistered transformation_manifest and were implemented in the execution pipeline; because no high-confidence flags occurred, no replacements were performed in this run (`analysis/contamination_summary.csv` empty). The preregistration additionally specified that medium-confidence flags would receive separate handling, but the primary confirmatory exclusion mechanism applied only to high-confidence flags.

Detector logging and artifact persistence policy. To bound artifact size the detector workflow was implemented to persist detailed per-item fuzzy overlap scores and full overlap tables only when a threshold crossing or replacement occurred (this behavior was documented in the preregistration detection_procedures). The rationale was to minimize unnecessary storage for large corpora while ensuring forensic traces for flagged events. As a result, in executions where no items met high-confidence thresholds the archive contains flagless contamination_summary artifacts but not a complete global table of per-item overlap scores. This implementation choice is recorded in the execution transformation manifest and explains why post-hoc aggregate overlap summaries are absent in the archive when no threshold crossings occurred.

Verifier and scoring rule (deterministic oracle). The ver-

ification oracle is `deterministic_netops_validator_v5` (`validator_version = 5.0`). For each candidate configuration the validator performs: parsing of the configuration commands into structured updates; simulated application to an explicit device/state model (`parse→state_update`); and property checks against the task's required_final_state and workflow constraints (reachability/preservation rules, protected interfaces, group constraints). The preregistered scoring rule `full_intent_constraint_validation` requires both (a) the final modeled state matches the expected_final_state on the allowed fields and (b) workflow constraints (sequence and transient constraints such as "must be down for field X") are respected. The validator emits binary pass/fail and detailed traces per episode: `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `violation_codes`, and `normalized_configuration`. These trace fields are archived for every episode and were the basis for the binary outcomes used in analysis (`execution/scoring/*.json`).

Analysis plan and confirmatory executor. The preregistration specified `paired_binary_analysis_v1` as the primary analysis executor. For the primary estimand (`paired_success_rate_difference_guarded_minus_baseline`) we use an exact McNemar two-sided test on the paired contingency table (`n11,n10,n01,n00`) and report a paired bootstrap percentile 95% confidence interval with 10,000 resamples (`bootstrap_seed = 1729`) as a descriptive companion. Secondary prespecified sensitivity analyses include: (i) treating failed model calls as failures (`complete_pair_primary_with_failure_as_zero_sensitivity`), (ii) fuzzy-threshold sensitivity (thresholds = 0.50, 0.65, 0.80), and (iii) subgroup analysis stratified by templated vs realistic items. The preregistration also contains power guidance showing that a two-call per item (independent generations) or a multi-model design increases power to detect modest paired differences (~4–6 percentage points under conservative assumptions) compared to the single-model shared-generation regime executed here.

Missingness, retries, and execution accounting. The preregistration specified a single automated model/API retry within 5 minutes for transient provider failures; if a retry failed the analysis plan treated the pair per the missingness contract and included a sensitivity scenario treating failed calls as failures. Execution logs show zero failed episodes and provider call accounting confirms `hosted_model_call_event_count = 150`, `completed_hosted_model_call_event_count = 150`, `failed_hosted_model_call_event_count = 0` (deterministic_reconciliation artifact). All seeds, provider traces, and validator traces were archived per the preregistered artifact list.

Implementation provenance and reproducibility. Key implementation artifacts frozen by preregistration and produced in execution include: `execution/model_configuration.json` (`declared_model_version = "gpt-5-mini"`, `independent_condition_generation = false`, `max_output_tokens = 2000`), `execution/task_manifest.jsonl` (deterministic generator seeds and task_payloads),

execution/raw_results.jsonl (shared and condition responses), and analysis artifacts (analysis/paired_contingency_table.csv, analysis/contamination_summary.csv). The preregistration and execution manifest SHA-256 values are recorded in the execution manifest and Disclosure Statement. The codepaths for detection, replacement, and validation are deterministic and archived; they can be re-run against different sampling or generation semantics (e.g., independent_condition_generation = true) to increase audit sensitivity without changing detectors or statistical estimands.

IV. RESULTS

Execution accounting and deterministic reconciliation. The archived execution manifest records `planned_episode_count` = 300 (150 items $\times$ 2 conditions) and `completed_episode_count` = 300 with `failed_episode_count` = 0. Because `independent_condition_generation` = false and no high-confidence exclusions occurred, the pipeline used `model_calls_used` = 150 (one shared generation per item) rather than two separate generations per condition; provider-call accounting and reconciliation artifacts confirm these counts and semantics (see execution/model_configuration.json, execution/task_manifest.jsonl, and analysis/deterministic_reconciliation.json). Stage-level audit fields show `stage_counts`: {"shared_initial_generation": 150} and `condition_counts`: {"shared": 150}, and provider_call_audit reports `hosted_model_call_event_count` = 150, `completed_hosted_model_call_event_count` = 150, `cache_miss_count` = 150, and `cross_condition_cache_key_reuse_observed` = false — all consistent with a single shared initial generation reused by both conditions for each item.

Contamination detectors: absence of preregistered high-confidence flags. The preregistered detectors produced zero high-confidence contamination flags for the executed sample; the analysis artifact analysis/contamination_summary.csv contains no flag rows. The detector workflow was implemented to persist per-item fuzzy overlap scores only when threshold crossings or replacements were required; because no items met the preregistered high-confidence threshold (exact OR 5-gram ≥ 0.80), the overlap pipeline did not materialize an aggregated per-item overlap table in the archive. This empty flag summary is an explicit artifact result (analysis/contamination_summary.csv) and not an absence inferred from silence: the detector logged no threshold events to persist. Consequently, no guarded exclusions or deterministic replacements were performed (`replacements_performed` = 0 in the execution manifest).

Validator outcomes and confirmatory statistics. The deterministic_netops_validator_v5 returned pass for all baseline episodes (`baseline_success_count` = 150/150; `baseline_success_rate` = 1.0) and for all guarded episodes (`guarded_success_count` = 150/150; `guarded_success_rate` = 1.0). The paired contingency table archived as analysis/paired_contingency_table.csv is `n11` = 150,

`n10` = 0, `n01` = 0, `n00` = 0. The primary estimand (`paired_success_rate_difference_guarded_minus_baseline`) = 0.0 percentage points. The preregistered paired bootstrap percentile 95% CI (10,000 resamples, `bootstrap_seed` = 1729) is [0.0, 0.0]; the exact McNemar two-sided test yields `p` = 1.0. These numbers are computed by the archived `paired_binary_analysis_v1` executor and are recorded in analysis/results.json and analysis/condition_summary.csv (`condition,successes,failures,observed,success_rate:baseline,150,0,150,1.0; guarded,150,0,150,1.0`).

Representative episode diagnostics and difficulty coverage. Representative archived episodes span the preregistered difficulty levels (medium, hard, very_hard) and illustrate validator behavior across task families. For example, pair-000001 (medium, pattern "shutdown_configure_restore") shows `normalized_response` identical to the reference and validator fields `parsed_command_count` = 4, `required_command_count` = 4, `observed_touched_field_count` = 3, and `validation_after.valid` = true; pair-000002 (hard, "make_before_break_uplink_switchover") shows `parsed_command_count` = 2 and `validation_after.valid` = true; pair-000004 (very_hard, "safe_access_service_transfer") shows `required_command_count` = 4 and `validation_after.valid` = true. These representative traces are included among the archived execution/scoring artifacts and demonstrate that the validator exercised per-task workflow constraints and recorded concrete parse $\rightarrow$ state metrics for inspection. Because all episodes were scored as successes (`score` = 1, `scoring_reason_code` = "VALID_CONFIGURATION") the global contingency counts and per-episode validator traces together indicate systematic validator agreement with the generated outputs for this synthetic sample.

Provider tracing, shared generation semantics, and implications. Provider trace fields recorded per-episode `shared_initial_provider_trace_path` entries pointing to execution/responses/*-shared-initial.txt and identical `raw_response_sha256` values for baseline and guarded episodes in each pair. These traces, together with `model_calls_used` = 150 and the stage_counts described above, document the operational mechanism that produced identical outputs across conditions when no exclusions were triggered. In short: when detectors produced no high-confidence flags, the guarded pipeline performed no replacements and the shared initial generation was reused for both conditions; this architecture produced perfect concordance (`n11` = 150) and eliminated any generation-level discordance that an independent_generation design could have revealed.

Missing overlap summaries and interpretation. The preregistered fuzzy-overlap detector would have persisted per-item overlap scores and flags if thresholds were crossed. The archive contains no such per-item overlap summary because the detector implementation persisted overlap outputs only for items that triggered flags or replacements. The absence of persisted overlap scores is therefore an explicit artifact of the detector workflow combined with zero threshold cross-

ings; it prevents reporting of min/median/percentile overlap statistics for this run, and we note this gap as an empirical limitation of the executed logging policy (see `analysis/contamination_summary.csv` and the `analysis_log.jsonl` entry describing detector events).

Synthesis of artifact evidence. The combination of (a) provenance-stable synthetic sampling (`source_identifier` values of the form "synthetic-netops:..." recorded in `execution/task_manifest.jsonl`), (b) shared_initial_candidate generation semantics (`execution/model_configuration.json`), (c) provider call accounting (`analysis/deterministic_reconciliation.json`), and (d) empty `contamination_summary.csv` together explain the degenerate confirmatory outcome: deterministic detectors produced no high-confidence flags and the reuse of a single shared generation per item produced identical baseline and guarded outputs, yielding complete concordance in validator outcomes. All numbers and traces cited above are archived in the execution bundle and referenced by the artifact paths given here (e.g., `execution/raw_results.jsonl`, `analysis/paired_contingency_table.csv`, `analysis/condition_summary.csv`, `analysis/contamination_summary.csv`, `analysis/deterministic_reconciliation.json`).

V. DISCUSSION

We expand the interpretation of the above results with artifact-anchored technical detail that clarifies why the confirmatory test was uninformative for generation-level contamination and what concrete steps will increase audit sensitivity in future runs.

Execution factors that produced the degenerate paired result. Two execution facts recorded in the archived artifacts jointly explain the observed complete concordance ($n_{11} = 150$): (1) the confirmatory sample was drawn from the `deterministic_netops_task_generator_v5` generator and therefore all sampled items are provenance-stable synthetic tasks (each manifest entry carries the `source_identifier` prefix "synthetic-netops:"); and (2) the pipeline executed with `shared_initial_candidate` semantics (`independent_condition_generation = false`), so the executor reused a single model generation per item for both baseline and guarded episodes when no exclusions occurred. Both facts are recorded in the execution artifact set (notably `execution/task_manifest.jsonl` and `execution/model_configuration.json`) and the `deterministic_reconciliation` artifact confirms that `model_calls_used = 150` rather than the preregistered two calls per item. Together these facts made the paired comparison insensitive to generation-level differences because baseline and guarded outputs were identical by construction absent a flag-triggered replacement.

Why per-item fuzzy overlap summaries are absent and the implications. The contamination detector implementation in the archived pipeline persisted detailed fuzzy-overlap outputs only when a tokenized 5-gram overlap exceeded the preregistered thresholds or when replacement handling

was invoked. The `contamination_summary` artifact (`analysis/contamination_summary.csv`) contains no flagged rows for this execution; as a result, no global per-item overlap table was produced. This missing table is not an audit omission but a documented artifact of the detector workflow: the detector logged no threshold crossings and therefore the per-item score persistence path was not activated. The consequence is practical: readers cannot inspect a distribution of overlap distances to the 0.80 threshold for these items from the archive. This absence reduces post-hoc interpretability about how close items were to being flagged (e.g., whether maximum overlaps were 0.10 vs 0.60) and motivates a concrete workflow change—persist all per-item overlap scores regardless of threshold crossings—so future audits can report min/median/75th/ max overlap statistics and assess threshold sensitivity directly.

Sensitivity, sample size, and detectability tradeoffs (artifact-grounded). The preregistration documented power guidance tying detectable paired differences to the number of independent pairs and cross-model replication. The executed design used one shared generation per item (`model_calls_used = 150`) rather than the two independent generations per item in the maximal preregistered plan (up to 300 calls). Reducing independent trials reduces statistical power to detect modest paired differences: with fewer independent generation events the probability of observing discordant pairs (n_{10} or n_{01}) falls even if contamination is present in a non-deterministic or paraphrase-sensitive manner. The archived `paired_bootstrap` CI and exact McNemar results (95% CI [0.0,0.0]; $p = 1.0$) accurately reflect this degenerate sample: when every pair is concordant the paired estimand is exactly zero and statistical tests correctly report no evidence of change. Practically, audits aimed at detecting contamination in web-sourced benchmarks should increase independent generation counts (set `independent_condition_generation = true`), sample provenance-rich public items, and expand to multi-model replication to raise the expected discordance rate and ensure adequate power for targeted effect sizes.

Design changes supported by archive evidence. Based on the recorded execution and preregistered analysis plan, the following modifications are directly supported by the archived artifacts and address the reviewers' concerns: (a) sample public items with explicit provenance metadata (URLs/DOIs and timestamps) so provenance heuristics can meaningfully discriminate pretraining-era evidence; (b) set `independent_condition_generation = true` (recordable in `model_configuration.json`) to ensure baseline and guarded outputs are independently realized, enabling observation of generation-level discordance; (c) persist per-item fuzzy 5-gram overlap scores in `analysis/contamination_scores.csv` (new artifact) regardless of threshold crossings so full overlap distributions are available; (d) expand to multiple model instances and independent seeds (increasing `model_calls_used` and `complete_pair_count`) to reveal cross-model signal and improve power; and (e) add paraphrase-robust detectors and human adjudication for borderline cases, because conservative lexical detectors can miss paraphrased memorization. Each

recommendation follows from explicit gaps visible in the current archived outputs: `model_calls_used = 150` (execution manifest), an empty `contamination_summary.csv`, and the lack of a per-item overlap table in analysis artifacts.

Operational implications and caution about generalization. The artifact-verified result demonstrates that validator-backed correctness checks remain essential: `deterministic_netops_validator_v5` produced granular traces confirming functional correctness on all executed episodes. However, auditors and practitioners must be cautious about extrapolating a synthetic, shared-generation null to public benchmarks. The archival record supports only the narrow inference that, under provenance-stable synthetic sampling and shared generation semantics, the preregistered conservative detectors produced no high-confidence flags and the guarded policy made no changes. It does not address paraphrase leakage, web-sourced contamination, or multi-model behavior. For operational NetOps risk assessments, audits should therefore combine provenance-rich sampling, independent generations, cross-model checks, and human adjudication to obtain evidence that generalizes to production corpora.

Summary and concrete next steps. The expanded artifact diagnosis clarifies the precise mechanism of degeneracy and prescribes artifact-anchored corrective steps for future audits. Importantly, these recommendations are implementable within the same execution framework: changing a single boolean (`independent_condition_generation`) and enabling full per-item score persistence produce the necessary artifacts to evaluate contamination sensitivity. Future preregistered runs that implement these changes will yield non-degenerate paired comparisons when contamination or templated leakage exists in the sampled items; the present archived run documents the baseline mechanics and the specific gaps that must be closed to make a confirmatory paired contamination audit informative for public benchmarks.

VI. LIMITATIONS

- Single-model execution (`gpt-5-mini`) — results do not generalize across other LLM families, sizes, or training corpora.
- Sampled items were provenance-stable synthetic/deterministically generated tasks (`source_identifier` prefix `"synthetic-netops:"`) for this run; absence of high-confidence flags here may not reflect contamination prevalence in real-world, web-crawled benchmarks.
- Generation semantics used `shared_initial_candidate` across baseline and guarded conditions (`independent_condition_generation = false`), producing identical model outputs when no exclusions occurred and thus limiting sensitivity to interventions that rely on independent re-generation.
- Contamination detection relied on preregistered conservative heuristics (exact match and fuzzy 5-gram overlap thresholds); subtler contamination patterns (paraphrased

memorization, partial leakage) may escape these heuristics and require richer provenance or human adjudication.

- Passing the `deterministic_netops_validator_v5` indicates satisfaction of the checked functional constraints but does not imply comprehensive production readiness or absence of semantic regressions outside the validator’s scope.

VII. CONCLUSION

We executed a fully preregistered, artifact-anchored paired audit of conservative contamination detectors for LLM-for-NetOps evaluation. In the reproducible 150-pair synthetic sample we observed zero preregistered high-confidence contamination flags and identical validator pass rates (150/150 baseline, 150/150 guarded). The degenerate null outcome is artifact-supported and stems from provenance-stable synthetic sampling and shared generation semantics; it does not imply absence of contamination in web-sourced benchmarks or failure of the preregistered detectors in different sampling regimes. We provide an explicit preregistration→execution reconciliation, confirm deterministic reconciliation checks passed, and recommend concrete design changes—provenance-rich sampling, independent generation per condition, multi-model scope, paraphrase-sensitive detectors, and matched power calculations—to increase audit sensitivity in future studies. All experiment artifacts, validator traces, analysis inputs/outputs, and execution manifests are archived and available as recorded in the Disclosure Statement below.

DISCLOSURE STATEMENT

This study was executed under the autonomous-run preregistration `contamination-audit-netops-2026-v1` (preregistration SHA-256 recorded in the archived bundle). LLM used: `gpt-5-mini` (`declared_model_version = "gpt-5-mini"`). Orchestration adapter: `hosted_netops_gvr_v1`. Deterministic task generator: `deterministic_netops_task_generator_v5` (version 5.0). Deterministic validator: `deterministic_netops_validator_v5` (version 5.0). Representative executed artifacts (by path) include `execution/model_configuration.json`, `execution/task_manifest.jsonl`, `execution/raw_results.jsonl`, `analysis/paired_contingency_table.csv`, and `analysis/contamination_summary.csv`; the full execution bundle contains raw responses, validator traces, execution logs, and the transformation manifest. Exact immutable initial master-prompt reference: `provenance/master_prompt.txt` (SHA-256: `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba`). Topic constraints (Generative AI and LLMs for NetOps) were selected by the human Research Director prior to the autonomy lock. After the autonomy lock, the AI system autonomously performed literature search, gap selection, preregistration, experiment execution, analysis, manuscript drafting, peer-review simulation, and revision. All generation prompts, model outputs, validator traces, sampling seeds, replacement rules, execution logs, and analysis artifacts were produced and archived by the autonomous pipeline and are included in

the execution bundle. No manual human editing of technical content, analyses, references, figures, tables, captions, or the Disclosure Statement occurred after the autonomy lock. Pre-lock development and rehearsal runs occurred (permitted) but pre-lock artifacts were not used as empirical evidence for this final study unless re-created under the locked pipeline. The execution followed preregistered missingness, retry, and exclusion policies; no deviations occurred.

REFERENCES

- [1] <https://arxiv.org/abs/2604.22513>
- [2] <https://doi.org/10.48550/arxiv.2606.06212>
- [3] <https://doi.org/10.48550/arxiv.2605.22092>
- [4] <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659>
- [5] <https://doi.org/10.1007/s11704-026-60308-3>
- [6] Buğra Alperen Uluirmak, Rifat Kurban, "EvalSafetyGap: A Hybrid Survey and Conceptual Framework for LLM Evaluation-Safety Failures", 2026
- [7] Nguyen Van Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, Ting Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions", 2024, 10.1145/3703155